

Análisis de Procedimiento Almacenado

sp_ResumenCarteraClientes — Tecnoglass / Energía Solar S.A. · Prueba Técnica Fullstack

1. Problemas críticos identificados

Problema	Ubicación	Impacto
Cursor externo sobre todos los clientes	DECLARE ClienteCursor	N iteraciones — una por cliente
Cursor interno por cada cliente (anidado)	DECLARE FacturaCursor	N × M iteraciones — O(N²) en el peor caso
SELECT TOP 1 dentro del cursor interno por cada factura	SELECT TOP 1 @UltimoEstado FROM HistorialFactura	N × M consultas individuales al historial
Sin índices implícitos — escaneo completo de tabla	WHERE Clienteld = @Clienteld / WHERE Facturald = @Facturald	Table scan repetido en cada iteración
Tabla temporal #Resumen con INSERT fila a fila	INSERT INTO #Resumen VALUES (...)	Overhead de I/O por cada cliente procesado

El patrón de dos cursores anidados más una subconsulta interna produce una complejidad asintótica **O(N × M × K)**, donde N = clientes, M = facturas por cliente, K = registros en HistorialFactura por factura. En bases de datos con miles de clientes y decenas de facturas cada uno, esto se traduce en cientos de miles de round-trips al motor de SQL Server, con bloqueos de lectura sostenidos que bloquean escrituras concurrentes.

2. Propuesta de optimización — enfoque Set-Based

La reescritura elimina todos los cursores y reemplaza las N × M operaciones fila por fila por dos operaciones de conjunto. El motor de SQL Server está optimizado para trabajar sobre conjuntos completos; los cursores fuerzan un plan de ejecución secuencial que descarta esa ventaja.

Pseudocódigo de la versión optimizada:

```

WITH UltimoEstado AS (
-- ROW_NUMBER reemplaza el SELECT TOP 1 dentro del cursor interno.
-- Una sola pasada sobre HistorialFactura para todos los clientes.
SELECT FacturaId, Estado,
ROW_NUMBER() OVER (PARTITION BY FacturaId
ORDER BY FechaMovimiento DESC) AS rn
FROM HistorialFactura
)
SELECT
f.ClienteId,
COUNT(*) AS TotalFacturas,
SUM(CASE WHEN ue.Estado = 'VENCIDA' THEN 1 ELSE 0 END) AS FacturasVencidas,
CAST( SUM(CASE WHEN ue.Estado = 'VENCIDA' THEN 1.0 ELSE 0 END)
/ NULLIF(COUNT(*), 0) * 100 AS DECIMAL(10,2)) AS PorcentajeMora
FROM Factura f
JOIN UltimoEstado ue ON ue.FacturaId = f.Id AND ue.rn = 1
GROUP BY f.ClienteId;
```

Comparación de enfoques:

Aspecto	SP original	SP optimizado
Complejidad	O(N × M × K) — cursores anidados	O(N + M) — dos JOINS set-based
Consultas al motor	1 por cliente + 1 por factura + 1 por historial	2 operaciones totales — CTE + GROUP BY

Aspecto	SP original	SP optimizado
Bloqueos	Shared locks durante todo el recorrido de cursores	Read committed snapshot o snapshot isolation
Concurrencia	Cursores abren transacciones implícitas largas	Operación atómica, libera recursos rápidamente
Mantenibilidad	80+ líneas con OPEN/FETCH/CLOSE duplicados	~18 líneas declarativas, fácil de auditar

3. Mejoras complementarias recomendadas

- **Índice cubriente en HistorialFactura:** CREATE INDEX IX_HF_FacturaId ON HistorialFactura (FacturaId, FechaMovimiento DESC) INCLUDE (Estado) — elimina el key lookup que generaría la CTE al resolver ue.Estado sin acceder a la tabla base.
- **Índice en Factura.ClienteId:** ya debería existir como FK; verificar que tenga estadísticas actualizadas para que el optimizador escoja Hash Match o Merge Join sobre el GROUP BY.
- **Snapshot Isolation:** habilitar READ_COMMITTED_SNAPSHOT en la base de datos para que lecturas analíticas como esta no compitan con escrituras transaccionales del sistema operativo.
- **SET NOCOUNT ON:** agregar al inicio del SP para suprimir mensajes de filas afectadas y reducir tráfico de red en cada operación interna.